RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable

Document number: P3911R1

Date: 2026-01-05

Authors: Darius Neațu <dariusn@adobe.com>,
Andrei Alexandrescu <andrei@nvidia.com>, Lucian Radu Teodorescu <lucteo@lucteo.ro>,
Radu Nichita <radunichita99@gmail.com>, Herb Sutter <herb.sutter@gmail.com>

Audience: EWG

Contents

Abstract

Revision History

Acknowledgments

0. TL;DR

1. Introduction

- 1.1 Overview of this R1 revision: Following EWG Kona 2025 direction

2. Scope

- 2.1 Design constraints per EWG mandate
- 2.2 Key open questions addressed by this paper

3. Proposal: Add source syntax for minimal always-enforced contract assertions

- 3.1 Motivation and existing practice

4. Design choice 1: Should "always-enforced" contract assertions **support `enforce` and/or `quick_enforce` semantics**?

- 4.1 Enforcement option 1: Support "always contract-terminate," and leave calling a violation handler implementation-defined

- 4.2 Enforcement option 2: Support both `enforce` and `quick-enforce` contract semantics

5. Design choice 2: What **syntax** should be used?

- 5.1 Syntax option 1: Use `<label>`
- 5.2 Syntax option 2: Use `!` (bang)
  - 5.2.1 `!` (bang) existing practice

6. Conclusion

7. Wording

8. Frequently Asked Questions

- 8.1 Why not a syntax such as `enforce_pre` or `pre_always`?
- 8.2 Why not use a terse syntax with a symbol other than `!`, such as `pre#` or `pre@`?
- 8.3 Why not do this only for `pre`, leaving `post` and `contract_assert` unchanged?
- 8.4 What about support for indirect calls (pointers to functions and virtual functions)?
- 8.5 How will this change evaluation semantics?
- 8.6 How does this proposal impact the Standard Library?
- 8.7 Does this paper address all the questions raised in P3912R0 [5], P3919R0 [6], and P3946R0 [7]?
- 8.8 What about requiring `#include <contracts>` (or `import std;`)?
- 8.9 Will "always-enforced" contract assertions behave differently from the "normal" ones?
- 8.10 What does it mean that "execution will not continue past the contract assertion" when an "always-enforced" contract assertion is violated?

9. References

Abstract

Building on motivation from EWG Kona 2025, this paper proposes a minimal pure extension to the C++26 Contracts facility. This extension addresses the Romanian National Body's (NB) concern by enabling the writing of an individual contract assertion that guarantees the

program execution will not continue past the contract assertion if it is violated, regardless of the semantics of other contract assertions. This addition to the C++26 Contracts facility allows reliable use of enforced contract assertions in large codebases with mixed contract semantics and in hardened libraries.

This paper's narrow extension is both necessary and sufficient to address the Romanian NB's concern. It does not preclude, nor co-require, proposals that add properties to "normal" or "always-enforced" contract assertions (such as no-constification, single evaluation, or similar properties) that may achieve consensus in EWG.

Revision History

- P3911R1 (http://wg21.link/P3911R1) (December 2025, after Kona Meeting)
  - Focus on implementing the EWG mandate: see §1.1 Overview of this R1 revision and sections §2-§5.
  - Add Acknowledgments section.
  - Add §0 TL;DR section.
  - Temporarily remove content in section 7. Wording until EWG agrees on design directions in telecons.
  - Add §8 Frequently Asked Questions section.

- P3911R0 (http://wg21.link/P3911R0) [1] (November 2025, Kona Meeting)
  - Surface the issue from Romanian NB comment RO 2-056 6.11.2 [basic.contract.eval] Make contract assertions reliably non-ignorable (https://github.com/cplusplus/nbballot/issues/631).
  - Iterate through all four possible solutions (labeled as v1 (https://wg21.link/P3911R0#v1-Add-pre-minimal-always-enabled-contracts), v2 (https://wg21.link/P3911R0#v2-Remove-ignore-Semantics), v3 (https://wg21.link/P3911R0#v3-Add-full-Contracts-Properties-Framework), v4 (https://wg21.link/P3911R0#v4-Move-C-Contracts-to-Other-Shipping-Vehicle)), which should be considered in EWG.

Acknowledgments

We thank everyone who provided feedback on drafts of this paper, in particular: Gašper Ažman, Joshua Berne, Peter Bindels, Timur Doumler, Andrei Durlea, Pablo Halpern, Dan Katz, Andrzej Krzemiński, Brad Larson, Lisa Lippincott, Jens Maurer, Jonas Persson, Oliver Rosten, David Sankel, Adrian Stanciu, Ville Voutilainen.

0. TL;DR

Currently, if a contract assertion predicate evaluates to `false`, evaluation semantics behave as follows:

- `ignore` continues execution;
- `observe` calls the handler and continues execution;
- `quick-enforce` terminates execution immediately;
- `enforce` calls the handler and terminates execution, unless an exception is thrown in the handler.

`pre`, `post`, and `contract_assert` can use any of these semantics. It is implementation-defined which evaluation semantics is used for each predicate evaluation.

We present two alternative additions to the C++26 Contracts facility:

1. Introduce "hardened" variations of `pre`, `post`, and `contract_assert` that may only use `quick-enforce` or `enforce` semantics. It is implementation-defined which of these two semantics is used for each predicate evaluation. Syntax: `pre<terminating>` or `pre!`.

2. Introduce "always-enforce" and "always-quick-enforce" variations of `pre`, `post`, and `contract_assert` that may only use `enforce` or `quick-enforce` semantics, respectively, for each predicate evaluation. Syntax: `pre<enforce>` or `pre!` for "always-enforce"; and `pre<quick_enforce>` or `pre!!` for "always-quick-enforce".

1. Introduction

This paper was prompted by the concern raised in RO 2-056 (https://github.com/cplusplus/nbballot/issues/631) that limiting C++26 Contracts to enforcement semantics chosen post-definition introduces a usability hazard: programmers may write a contract assertion under an expectation of enforcement that is not guaranteed. Users who need "always-enforce" contract assertions are forced to duplicate logic between contract assertions and regular code.

The initial (P3911R0 (http://wg21.link/P3911R0) [1]) version of this paper discussed four potential solution approaches:

1. Add a way to write a contract assertion that will always be enforced in all cases (regardless of the currently active contract semantics).
2. Remove "ignore" (and, as was intended, "observe") contract semantics from the C++ Working Draft.
3. Adopt all of P3400R2 Controlling Contract-Assertion Properties (https://wg21.link/P3400R2) [3] — the full C++ Contracts properties framework.
4. Remove the C++26 Contracts facility from the C++ Working Draft introduced by P2900R14 (https://wg21.link/P2900R14) [2].

The R0 revision of this paper recommended pursuing option 1. It is purely additive to C++26 Contracts, and pulls forward into C++26 one specific capability from the post-C++26 direction described in P3400 that addresses the current concerns, and does so in a forward-compatible way with P3400.

1.1 Overview of this R1 revision: Following EWG Kona 2025 direction

In Kona 2025, EWG encouraged work (https://github.com/cplusplus/nbballot/issues/631#issuecomment-3488277609) in the direction proposed in D3911R0 (now P3911R0 (https://wg21.link/P3911R0) [1]):

> Poll: RO 2-056 6.11.2 [basic.contract.eval] Make contracts reliably non-ignorable — we want to pursue a change for C++26 as proposed in one of the four options listed in D3911R0.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 18 | 25 | 22 | 7 | 0 |
>
> Result: consensus for change

Option 1, which proposes making contracts reliably non-ignorable by adding a syntax that ensures contract assertions have guaranteed effects, was the main focus of the EWG discussion.

EWG raised three important additional observations regarding Option 1:

- It is **a pure extension** that does not affect existing C++26 Contracts semantics and pulls forward a **specific narrow capability** already considered and encouraged in P3400 for post-C++26 Contracts evolution.
- It may also **address other NB concerns**, notably enabling the implementation of the C++26 hardened standard library using C++26 Contracts. (For example, via `ifdef` guards, such as `#ifdef __hardening` / `pre<terminating>(idx < size())` / `#endif`; today, libraries that do this put nonstandard code in the guarded section.)
- It should apply uniformly to **all three contract assertions** — `pre`, `post`, and `contract_assert` — as requested by several EWG participants. This ensures consistent capability across all contract assertion types.

2. Scope

2.1 Design constraints per EWG mandate

To follow EWG direction, increase consensus, and avoid scope creep, this revision adopts the following design constraints:

1. **Scope: Option 1 only.** We have removed options 2, 3, and 4 in accordance with EWG guidance and to increase consensus on the C++26 Contracts feature set.

2. **Pure extension.** This paper proposes a pure extension to the C++26 Contracts facility from the post-Kona C++ Working Draft. It does not change the existing semantics of `pre(condition)`, `post(ret: condition)`, or `contract_assert(condition)`.

3. **Minimal semantics delta.** This paper does not propose additional changes beyond providing a way to spell "always-enforced" contracts assertions in source code, which is enough to satisfy the Romanian NB's concern and which EWG encouraged. To avoid scope creep, it does not propose additional properties of "always-enforced" contract assertions (e.g., no-constification or single-evaluation) that differ from existing C++26 Contracts and may achieve EWG consensus independently. The Romanian NB is neither in favor of nor opposed to such additional properties.

See §8 Frequently Asked Questions for further discussion of additional design questions.

2.2 Key open questions addressed by this paper

We identified the following open questions:

Q1. Should "always-enforced" contract assertions **support `enforce` and/or `quick_enforce` semantics**? See section §4.
Q2. What **syntax** should be used? See section §5.

3. Proposal: Add source syntax for minimal always-enforced contract assertions

This paper proposes introducing a **minimal syntax to mark certain contract assertions as "always-enforced" in source code (see §4)**. This additional syntax is placed between the contract keyword and the condition (see §4 and §5 for the proposal):

> Unchanged syntax : `pre (x != 0)`
> Added "always-enforced" syntax(es): `pre` *placeholder-tokens* `(y != 0)`

A "`pre` *placeholder-tokens* `(condition)`" is always enforced, independent of any other contract semantics selected for `pre (condition)`.

Note that the same syntax choice applies to `post` and `contract_assert`.

This design represents a balanced compromise: it preserves configurability while ensuring selected contract assertions are reliably checked. No future extensions are precluded.

3.1 Motivation and existing practice

There is a substantial body of existing practice that motivates "always-enforced" contract assertions, demonstrates their utility, and makes the model familiar to users. Many production codebases ship an always-on assertion (separate from `assert`, which is sometimes-on by means of the `NDEBUG` macro).

Such mechanisms, commonly named `CHECK`, `VERIFY`, or `ALWAYS_ASSERT` in existing codebases, provide run-time validation of key conditions even in production: they verify the condition and do not continue execution if it does not hold. They are widely used as safety checks for preconditions, i.e., as intentionally redundant code that has no observable effect in correct program executions, but deterministically prevents execution from continuing in incorrect executions. Such mechanisms may also be used outside an assertion context, where they are not redundant.

Below, we present a selection of representative, widely used examples that leverage non-continuing (terminating) enforcement semantics in production:

- **Google Abseil**'s CHECK (https://abseil.io/docs/cpp/guides/logging#CHECK) ("terminates the process" in all builds; paired with debug-only `DCHECK`) is used in a plethora of Google and third-party projects, notably including **Protocol Buffers**, **glog**, **gRPC**, **Chromium**, and **TensorFlow**.
- **Facebook Folly**'s XCHECK (https://github.com/facebook/folly/blob/main/folly/logging/xlog.h) (always-on - "this crashes the program").
- **WebKit**'s RELEASE_ASSERT (https://github.com/WebKit/WebKit/blob/main/Source/WTF/wtf/Assertions.h) ("crashes with info"; paired with `ASSERT`).
- **LLVM**'s cantFail (https://llvm.org/doxygen/llvm_2Support_2Error_8h.html) (several overloads - "report a fatal error").
- **Mozilla Gecko**'s MOZ_RELEASE_ASSERT (https://github.com/mozilla/gecko-dev/blob/5836a062726f715fda621338a17b51aff30d0a8c/mfbt/Assertions.h#L566).
- **Apache Arrow**'s ARROW_CHECK (https://github.com/apache/arrow/blob/3dc982183db45ced57ef1565e022dcb647a00745/cpp/src/arrow/util/logging.h#L67).

Note: The quoted phrases above are taken verbatim from the respective project documentation.

Experience shows that defining a project-specific assertion facility (or using an established one such as Abseil or Folly) is a mark of project growth and maturity. Such facilities almost always include an API that deterministically terminates execution when a boolean condition evaluates to false.

Externally-controlled assertions and always-active checks rely on the same underlying mechanics, which is why mature codebases commonly pair them: a debug-only assertion alongside an always-fatal check for essential invariants. Absent a reliable facility that terminates execution when a condition is false, users must duplicate logic by first writing an assertion that may or may not be active, then adding manual code to test the condition again and terminate on failure. A single, always-available check facility avoids that duplication, keeps intent obvious, and reduces divergence across configurations.

4. Design choice 1: Should "always-enforced" contract assertions support `enforce` and/or `quick_enforce` semantics?

For this design choice, the following major options were considered:

- Support only one of the "enforce" or "quick-enforce" contract semantics?
- Support both explicitly selectable in the source code?
- Do what a hardened standard library does (i.e., leave it implementation-defined)?

During the EWG discussion and on-time papers in Kona, such as P3835R0 (http://wg21.link/P3835R0)[4], it appears there are three large, partially overlapping groups:

- those who want to specify "always enforce" semantics in source code;
- those who want to specify "always quick-enforce" semantics in source code; and
- those who want to specify "never ignore or observe, without a preference for enforce or quick-enforce" contract semantics in source code.

Either option presented in sections §4.1 or §4.2 (see below) would satisfy the Romanian NB's concern. More generally, if EWG identifies another option with consensus support that guarantees program execution does not continue (see §8.10) when that contract assertion is

violated (including, but not limited to, never being subject to ignore or observe semantics), the Romanian NB would accept that as satisfactory.

4.1 Enforcement option 1: Support "always contract-terminate", and leave calling a violation handler implementation-defined

This option makes **a single "always contract-terminate" syntax** selectable in source code, leaving it implementation-defined whether a contract violation handler is invoked.

See §5 for *placeholder* syntax options.

> - Unchanged : `pre (x != 0)`
> - Always contract-terminated: `pre` *terminate-tokens* `(y != 0)`

A "`pre` *terminate-tokens* `(condition)`" shall always be enforced regardless of other semantics that may be used for `pre (condition)`, and the implementation shall define whether or not a violation handler is called (a terminating semantics is guaranteed to be used).

Note that the same design choice applies to `post` and `contract_assert`.

4.2 Enforcement option 2: Support both `enforce` and `quick_enforce` contract semantics

This option provides **two distinct placeholders**, allowing the programmer to express **either "always enforce" or "always quick-enforce"** in source code for a given contract assertion.

> - Unchanged : `pre (x != 0)`
> - Always enforced : `pre` *enforce-tokens* `(y != 0)`
> - Always quick-enforced: `pre` *quick-enforce-tokens* `(y != 0)`

A "`pre` *enforce-tokens* `(condition)`" is always enforced, and a "`pre` *quick-enforce-tokens* `(condition)`" is always quick-enforced, regardless of any other contract semantics selected for `pre (condition)`.

Note that the same design choice applies to `post` and `contract_assert`.

5. Design choice 2: What syntax should be used?

Several alternatives were considered. This paper proposes two concise syntax choices (§5.1 or §5.2 - see below) that are forward-compatible with future directions, such as P3400R2 (https://wg21.link/P3400R2), by requiring `#include <contracts>` (or `import std;` - see §8.8).

Either option presented in sections §5.1 or §5.2 (see below) would satisfy the Romanian NB's concern. More generally, if EWG identifies another option that has consensus support, the Romanian NB would accept that, provided that the constract semantics guarantee the program code does not continue execution after a violation (see sections §4 and §8.10).

The Romanian NB requests only a narrow extension that allows a contract assertion to be marked as enforced in source code. Instead of inventing a new syntax, this paper follows the design direction previously discussed and encouraged in EWG.

5.1 Syntax option 1: Use `<label>`

This option follows the current syntax of proposed P3400R2 (https://wg21.link/P3400R2).

Under **Enforcement option 1** (see §4.1), use `<terminating>` (or other similar label - e.g., `<always_terminating>`, `<hardened>` etc) to express a contract assertion that is **always** evaluated using terminating semantics (per §4.1, with violation-handler invocation left implementation-defined):

```
pre <terminating> (condition)
post <terminating> (ret: condition)
contract_assert <terminating> (condition)
```

Under **Enforcement option 2** (see §4.2), which provides two distinct syntactic forms, use the labels `<enforce>` and `<quick_enforce>` (or other similar labels - e.g., `<always_enforce>` and `<always_quick_enforce>`, etc) to express a contract assertion that is **always** evaluated using the "enforce" and "quick-enforce" semantics, respectively:

```
pre <enforce> (condition)
post <enforce> (ret: condition)
contract_assert <enforce> (condition)

pre <quick_enforce> (condition)
post <quick_enforce> (ret: condition)
contract_assert <quick_enforce> (condition)
```

5.2 Syntax option 2: Use ！ (bang)

This option follows the original strawman syntax in R0 of this paper (see P3911R0 (http://wg21.link/P3911R0)).

Under **Enforcement option 1** (see §4.1), use ！ to express "terminating" semantics (with violation-handler invocation left implementation-defined):

```
pre ! (condition)
post ! (ret: condition)
contract_assert ! (condition)
```

Under **Enforcement option 2** (see §4.2), which has two syntaxes, use ！ and ！！ to express a contract assertion that is evaluated using the "enforce" and "quick-enforce" semantics, respectively:

```
pre ! (condition)
post ! (ret: condition)
contract_assert ! (condition)

pre !! (condition)
post !! (ret: condition)
contract_assert !! (condition)
```

We believe this style is forward-compatible with P3400 because "terminating" semantics is likely a common use case that warrants a convenient shorthand; if we had P3400 today, "always enforce/quick-enforce this contract assertion" would still benefit from a dedicated terse shorthand to keep common usage simple and ergonomic.

5.2.1 ！ (bang) existing practice

This follows existing use of the `!` suffix to connote the sense of "do not continue execution" (with halt or throw semantics) in other popular languages that also use `!` as a prefix logical-not operator, and this notation is widely understood and does not confuse programmers. For example:

- TypeScript: `value!.prop` is a non-null assertion
- Swift: `let x = optional!` is a forced unwrap with assert semantics, where if the optional is `nil` the program terminates
- Kotlin: `val x = nullable!!` is a non-null assertion
- Elixir: `File.read!/1` is a "bang function" that raises an exception on error
- Ruby: `update!` and `save!` are idiomatic names for methods that mutate state or raise on failure

The following languages use `!` as a prefix logical-not operator and also use `!` as suffix with a distinct, unrelated meaning, again without causing confusion:

- Julia: `sort!(v)` is a mutating-function convention
- Rust: `println!("hi")` is a macro invocation, and `!` is a "never" type (when written in a grammar position where a type would appear)
- D: `Tuple!(int, string)` is a template instantiation

A concern has been raised that `!` might be interpreted as "logical NOT this precondition." This concern has two aspects:
- That `!` as a prefix and as a suffix have different meanings. Evidence from experience with the languages listed above suggests this is not a significant problem.
- That this specific option permits writing `pre!(condition)` and `pre(!condition)` with distinct meanings. We note that C++ permits syntax such as `if (!x > 1)` and `if (!(x > 1))` without this being a known source of bugs as reflected in books, articles, "gotcha" lists, and coding standards documents.

One drawback is that this allows writing `pre not` using an alternative token. We note that C++ already has several precedents: the alternative token `compl` for `~` can be used in the destructor declaration syntax; the alternative token `bitand` for '&' can be used in reference declarators and as an address-of operator; and the alternative token `and` for '&&' can be used to construct rvalue reference types. Spelling a destructor declaration as `compl Type()`,

a function declaration as `void f(int and a, int bitand b)`, or the address of an object as `bitand a` (see https://godbolt.org/z/x7EY1Eoh3 (https://godbolt.org/z/x7EY1Eoh3)) may be an amusing curiosity but not a source of practical problems.

6. Conclusion

This paper proposes a small, purely additive extension to the C++26 Contracts facility that allows a contract assertion to be marked as "always- enforced". By enabling a simple source-level choice among `enforce`, `quick-enforce`, or `terminating` semantics, the proposal improves the practical reliability of contract assertions in large codebases and hardened libraries, while preserving the existing design and helping build consensus around the C++26 Contracts facility and the C++ Working Draft.

7. Wording

TBD — R0 had initial wording, which is now removed; we'll add an updated one when EWG decides about the final design (multiple design options are presented in this paper).

8. Frequently Asked Questions

8.1 Why not a syntax such as `enforce_pre` or `pre_always`?

Several syntax alternatives were considered.

This paper proposes `pre<label>` style and `pre!` style because these two syntaxes are **long-term forward-compatible with P3400**, which has already been encouraged in EWG, and therefore aligns with the EWG Kona 2025 direction to bring forward to C++26 this narrow previously proposed capability.

Name-based syntaxes such as `enforce_pre` or `pre_always` would not naturally compose with future generalized contract assertion annotations envisioned by P3400.

- The `pre<label>` style is directly forward-compatible with P3400 because it follows the same syntax direction. If P3400 were available today, this is the syntax that would be used.
- The `pre!` style is also forward-compatible with P3400 because always enforcing is expected to be a common use case that warrants a concise shorthand; even if P3400

were available today, "always enforce this contract assertion" would still benefit from a dedicated, terse shorthand.

This paper does not propose alternative syntaxes because they would not be forward-compatible with P3400, which aligns with EWG guidance for this proposal. If P3400 were available, EWG would be unlikely to adopt an additional spelling that would be redundant and as verbose as `pre<label>`.

However, this proposal is open to adopting any syntax choice that EWG determines increases consensus.

8.2 Why not use a terse syntax with a symbol other than `!`, such as `pre#` or `pre@`?

The `!` suffix follows successful existing practice in other languages, where it is widely recognized as indicating asserted or checked operations (see discussion in §5.2). Other symbols are less familiar and pose a higher risk of syntactic or lexical conflicts, and therefore are not proposed here.

However, as mentioned, this proposal is open to adopting any syntax choice that EWG recommends.

8.3 Why not do this only for `pre`, leaving `post` and `contract_assert` unchanged?

Consistency reasons.

Limiting always-enforced contract assertions to `pre` conditions would **leave gaps in reliability** because `post` conditions and `contract_assert` could still be ignored if different semantics are chosen separately. By applying the same approach across all contract assertion types, we ensure **consistent enforcement**, prevent silent failures, and simplify reasoning about contract assertions in large codebases.

However, we are not opposed to proceeding with a smaller subset if that increases consensus in EWG.

8.4 What about support for indirect calls (pointers to functions and virtual functions)?

This paper does not address those features because they are outside the scope identified by EWG for this proposal and no EWG-approved design exists for them at this time.

This proposal adds no constraints beyond those already present in C++26 that would limit the design of future features. For example, C++26 Contracts already permit `pre` contract assertions to be enforced; this proposal merely allows that choice to be made at the level of an individual contract assertion.

8.5 How will this change evaluation semantics?

This proposal has minimal impact on evaluation semantics.

When evaluating a contract assertion, the current C++26 Working Draft specifies that “it is implementation-defined which evaluation semantics is used for any given evaluation of a contract assertion” ([basic.contract.eval] p2 (https://eel.is/c%2B%2Bdraft/basic.contract#eval-2)). This proposal allows users to specify, in code, the evaluation semantic for targeted contract assertions. For example, users can write `pre!` to specify that a precondition is evaluated using terminating evaluation semantics.

The C++26 Working Draft also has a recommended practice for implementations to provide options to use ignore semantics for all contract assertions. With this proposal, this recommended practice is limited to contract assertions that are not always enforced.

8.6 How does this proposal impact the Standard Library?

No impact.

Users are not expected to need to distinguish between "normal" contract assertions and "always-enforced" contract assertions. For instance, when `x == 0`, `pre(x > 0)` and `pre!(x > 0)` would be reported with identical metadata to the contract violation handler.

8.7 Does this paper address all the questions raised in P3912R0 [5], P3919R0 [6], and P3946R0 [7]?

Yes, we think so.

Most of these questions are explicitly addressed in this paper, either in the proposal sections (see §3, §4, and §5) and/or in the FAQ section (see §8).

The remaining questions are implicitly addressed by the paper's deliberately narrow scope: it proposes adding only syntax to select the existing enforce or quick-enforce semantics for an individual contract assertion (semantics already permitted by the current C++ Working Draft for any contract assertion), and it proposes no other changes or extensions to the C++26 Contracts facility.

However, we are not opposed to considering other design choices proposed by EWG.

8.8 What about requiring `#include <contracts>` (or `import std;`)?

Yes, we think it should be a required dependency.

We think that any syntax proposal discussed in this paper and in EWG should require `#include <contracts>` (or `import std`). That way, future introduction of syntaxes (e.g., P3400-like syntaxes) does not constitute a breaking change, as they may also require that header.

8.9 Will "always-enforced" contract assertions behave differently from the "normal" ones?

No, except for the evaluation semantics (see §8.5).

This proposal does not change how contract assertions are evaluated. In particular, it does not affect how the predicate of a contract assertion is evaluated (how many times it is evaluated, whether it is evaluated on the caller or callee side, constification, etc.).

All rules from the existing C++26 Contracts facility apply for "normal" and "always-enforced" contract assertions the same way, except for the evaluation semantics (see §8.5).

8.10 What does it mean that "execution will not continue past the contract assertion" when an "always-enforced" contract assertion is violated?

In this paper, "execution will not continue past the contract assertion" means that execution does not proceed past the violated contract assertion (e.g., not calling guarded function for `pre! / pre<terminating>` precondition).

Does this proposal guarantee program termination in the event of "always-enforced" contract assetions violation?
No. This proposal *does not mandate program termination*; this proposal mandates using **only** `terminating semantics` for "always-enforced" contract assertions (which in most cases also means "contract-terminate"):

- For "always-enforced" contract assertions with `quick_enforce` semantics, a violation results in "program is contract-terminated" as per [basic.contract.eval#10] (https://eel.is/c++draft/basic.contract.eval#10).
- For "always-enforced" contract assertions with `enforce` semantics, a violation results similarly, by default, in "program is contract-terminated" as per [basic.contract.eval#13] (https://eel.is/c++draft/basic.contract.eval#13).
    - Do note that **this does not apply** when having an uncaught exception from the evaluation of the predicate or from a user-installed contract-violation handler that transfers control by throwing an exception.

In both cases, execution of the program after the violated contract assertion is prevented, and the asserted condition is guaranteed to hold for any code executed subsequently. See [basic.contract.eval#16] (https://eel.is/c++draft/basic.contract.eval#16) for details: "*The terminating semantics terminate the program if execution would otherwise continue normally past a contract violation*".

This distinction is intentional and aligns with the existing C++26 Contracts model, while still providing a reliable mechanism for specifying "always-enforced" contract assertions.

9. References

[1] P3911R0 — Darius Neațu, Andrei Alexandrescu, Lucian Radu Teodorescu, Radu Nichita, "RO 2-056 6.11.2 [basic.contract.eval] Make Contracts Reliably Non-Ignorable", WG21. http://wg21.link/P3911R0 (http://wg21.link/P3911R0) (published: 2025-11-04)

[2] P2900R14 — Joshua Berne, Timur Doumler, Andrzej Krzemieński, et al., "Contracts for C++", WG21. https://wg21.link/P2900R14 (https://wg21.link/P2900R14) (published: 2025-02-13)

[3] P3400R2 — Joshua Berne, "Controlling Contract-Assertion Properties", WG21. https://wg21.link/P3400R2 (https://wg21.link/P3400R2) (published: 2025-12-14)

[4] P3835R0 — John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, "Contracts make C++ less safe – full stop!", WG21. https://wg21.link/P3835R0 (https://wg21.link/P3835R0) (published: 2025-09-03)

[5] P3912R0 — Timur Doumler, Joshua Berne, Gašper Ažman, Oliver Rosten, Lisa Lippincott, Peter Bindels, "Design considerations for always-enforced contract assertions", WG21. https://wg21.link/P3912R0 (https://wg21.link/P3912R0) (published: 2025-12-15)

[6] P3919R0 — Ville Voutilainen, "Guaranteed-(quick-)enforced contracts", WG21. https://wg21.link/P3919R0 (https://wg21.link/P3919R0) (published: 2025-11-05)

[7] P3946R0 — Andrzej Krzemieński, "Designing enforced assertions", WG21. https://wg21.link/P3946R0 (https://wg21.link/P3946R0) (published: 2025-12-14)